The R1 Recipe in Code

Module 3 — Five stages, five tiny notebooks

One toy task — adding two small integers — runs through every stage.
Watch the same metric improve.

Stage 1 — Pre-training

> Predict the next token. That's it.

Demo →

`02_stage1_pretraining.ipynb`

A 2-layer transformer learns tinyshakespeare in 200 steps.

Stage 2 — Cold-start SFT

> Teach the format: `<think>…</think><answer>…</answer>`.

Demo →

`03_stage2_cold_start_sft.ipynb`

Fine-tune `gpt2` on 20 hand-written examples.

Stage 3 — RL with GRPO

> Sample G completions, reward correct ones, push the model toward the group's best.

No critic. No human labels. Just a checker.

Demo →

`04_stage3_rl_grpo_from_scratch.ipynb`

GRPO in ~60 lines of PyTorch on 1-digit addition.

Stage 4 — Rejection-sampling SFT

> Use the RL'd model as a data factory: generate, filter, fine-tune.

Demo →

`05_stage4_rejection_sampling_sft.ipynb`

Pass-rate jumps again with no new training signal — just better data.

Stage 5 — Distillation

> Compress the recipe into a smaller model with KL on logits.

Demo →

`06_stage5_distillation.ipynb`

A 2-layer student matches a `gpt2` teacher at a fraction of the params.

Recap

Stage	What it adds
Pre-train	Language
Cold-start	Format
RL (GRPO)	Skill
Reject-SFT	Stability
Distill	Efficiency

That is the whole R1 recipe.

This same recipe — GRPO + RL from verifiable rewards — is still how 2026 frontier reasoning models (GPT-5.5, Claude Opus 4.7, Gemini 3.1) are trained: scaled, not replaced.